A Minor Project Proposal Report on

RealSight - AI Generated Content Detection System

Submitted in Partial Fulfillment of the Requirements for the Degree of
Computer Engineering under Pokhara University

Submitted by:

Upendra Prasad Pant, Roll No. 45
Purshottam Jha, Roll No. 29
Manjan Shahi, Roll No. 25
Lokendra Prasad Sah, Roll No. 23

Under the supervision of
Manil Vaidhya

Date:
20-02-2082

Department of Computer Engineering
NEPAL COLLEGE OF INFORMATION TECHNOLOGY
Balkumari, Lalitpur, Nepal.

ABSTRACT

Rapid development in artificial intelligence (AI) has led to a rise in AI-generated content, making it increasingly difficult to differentiate between machine-generated and human-generated content. The project aims to address this issue by developing a cross-platform mobile application capable of identifying whether text or images are original or AI-generated. Users can paste text, upload a PDF, extract text via OCR from images, take a photo, or upload existing photos for analysis. The app uses custom AI models, which were trained with the use of Python, to analyze both text and image inputs. The tool aims to provides users with in-depth detection reports, such as accuracy scores and clear explanations for each result. The project hopes to provide a user-friendly, transparent, and reliable tool for navigating the issues of AI-generated content in the internet era.

Keywords: *AI-Generated Content Detection, text analysis, image authenticity, python, mobile app*

TABLE OF CONTENTS

1. INTRODUCTION……...……..1

1.1 PROBLEM STATEMENT……..1

1.2 PROJECT OBJECTIVES ……...1

1.3 SCOPE AND LIMITATION…….…….….2

2. LITERATURE REVIEW.....…..... 3

3. METHODOLOGY.....…..... 5

3.1 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) ……...5

3.2 DATA COLLECTION…….6

3.3 MODEL DEVELOPMENT…….7

3.4 APP DEVELOPMENT…….……...8

3.5 INTEGRATION AND TESTING……9

3.6 USER INTERFACE AND EXPERIENCE…….……9

3.7 FIGURES AND TABLES……...10

4. PROPOSED DELIVERABLES.....…..... 12

5. PROJECT TASK AND TIME SCHEDULE.....….....….... 13

5.1 PROJECT TASKS…….….13

5.1. GANTT CHARTS.....….....….... 14

6. REFERENCES.....….....….…. 15

LIST OF FIGURES

Figure 1: SDLC with Scrum…….5

Figure 2: System Architecture ……..…10

Figure 3: Use Case Diagram…….…...11

Figure 4: Class Diagram…….…….12

Figure 5: Sprints Timeline Gantt Chart……...……14

1. INTRODUCTION

Artificial intelligence (AI) has revolutionized content creation, producing increasingly sophisticated synthetic text and images that blur the line between what is real and what is not. This proposal outlines the development of a cross-platform mobile application that is intended to capture this dilemma by examining AI-created content in real-time: users can enter text by copying, upload PDFs, scan images using OCR, or take/upload photographs for analysis. Custom AI models will evaluate each submission and give a detailed report with confidence ratings and explicit reasoning on the detection process, hence offering a strong yet accessible tool for guaranteeing digital trust in the pervasive age of AI-generated media.

1.1 PROBLEM STATEMENT

The indistinguishability of AI-generated content from human work raises concerns. Detection tools, which are often web-based and lack mobile accessibility, frequently misidentify non-native authors' original work as AI-generated, resulting in unfair consequences. They also fail to detect sophisticated AI-generated content, allowing misinformation to spread, and provide no explanations for their classifications, leaving users uncertain about the authenticity of the information.

1.2 PROJECT OBJECTIVES

- To create an accessible mobile application capable of identifying AI-generated texts and images.
- To assist people in distinguishing between real and deceptive content while fostering trust, accuracy, and awareness around digital media.

1.3 PROJECT SCOPE AND LIMITATION

1.3.1 SCOPE

- Supports text and image analysis
- Uploading or snapping images with their camera
- Supports popular image formats like JPEG and PNG.
- Delivers clear, precise, and easy-to-understand results, indicating how likely it is that the content was AI-generated.
- Text extraction from image to perform text analysis.
- Upload PDF file for text analysis

1.3.2 LIMITATION

- No multilingual support (only English).
- No support for videos and audio.
- It can’t analyze content in live streams or real-time social media updates.

2.LITERATURE REVIEW

The rapid development of artificial intelligence (AI) has led to a dramatic increase in AI-generated content, encompassing text, images, and videos. This surge presents significant challenges in distinguishing between human and machine-generated content, particularly in academic environments where it threatens academic integrity through plagiarism and misinformation [1], [2]. Recent studies have assessed the performance of AI content detection tools, such as those from OpenAI, Copyleaks, GPTZero, and CrossPlag. For instance, Elkhatat (2023) evaluated these tools using paragraphs generated by ChatGPT models 3.5 and 4, finding higher detection accuracy for GPT-3.5 but noting persistent false positives with human-written content, especially from non-native English writers [3]. A comprehensive survey on large language model (LLM)-generated text detection further revealed that while these tools have advanced, they struggle with out-of-distribution issues and vulnerabilities to paraphrased or edited AI content [4]. Another study reported detection accuracy rates ranging from 55.29% to 97.0%, underscoring the inconsistency of current tools [5]. These findings highlight the need for more reliable and robust detection methods that deliver trustworthy results.

In the realm of mobile applications, the availability of tools for detecting AI-generated content remains limited. Web-based platforms such as Winston AI, Originality.ai, and Scribbr provide advanced detection capabilities, yet their presence as mobile apps is scarce, reducing convenience for users who primarily use mobile devices [6], [7]. For example, Copyleaks claims over 99% accuracy in identifying AI-generated text but operates predominantly as a web-based service without a dedicated mobile app [8]. Similarly, Grammarly's AI detection features, while integrated into its platform, lack specific mobile functionality [9]. This gap in mobile accessibility presents an opportunity for a mobile app that leverages machine learning to analyze both text and images, offering a user-friendly solution to detect AI-generated content.

To facilitate the development of such an app, relevant datasets are available for training detection models. The "CIFAKE: Real and AI-Generated Synthetic Images" dataset, introduced by Bird and Lotfi (2023), contains real and AI-generated images, making it a valuable resource for training models to identify synthetic visual content [10]. Additionally, the "LLM - Detect AI Generated Text Dataset" provides text samples specifically curated for training models to detect AI-generated text [11]. These datasets are critical for building accurate and effective detection algorithms.

The proposed mobile app aims to address these shortcomings by providing a platform capable of extracting text from images, supporting large image formats such as JPEG and PNG.

3. METHODOLOGY

The methodology for developing a cross-platform mobile application to detect AI-generated text and images is designed to ensure accuracy, usability, and reliability. The app, built using Flutter for Android and iOS compatibility, will enable users to paste text, upload PDFs, upload or capture images, and extract text from images via Optical Character Recognition (OCR). A detailed report will provide detection results, including accuracy, confidence scores, and explanations. The development process follows the Agile Scrum Software Development Life Cycle (SDLC) model, incorporating data collection, model development, app development, integration and testing, and user interface design. Below are the detailed steps involved.

3.1 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) WITH AGILE SCRUM

The project adopts the Agile Scrum methodology to facilitate iterative development, collaboration, and adaptability to changing requirements. Scrum's iterative nature, with short development cycles called sprints, ensures continuous feedback and improvement, making it ideal for a project with evolving user needs and complex technical requirements.

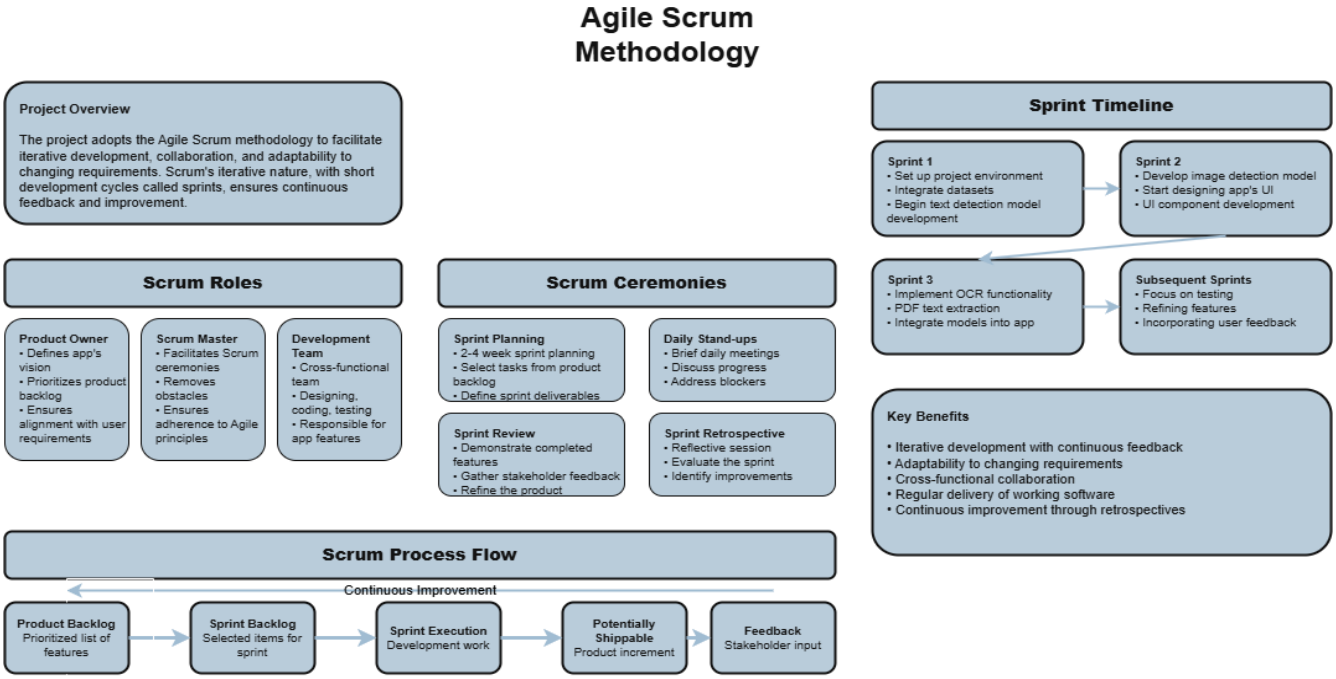

Figure 1: Agile(Scrum) for SDLC

The project will be divided into multiple sprints, each focusing on specific deliverables:

- **Sprint 1:** Set up the project environment, integrate datasets, and begin text detection model development.
- **Sprint 2:** Develop the image detection model and start designing the app's user interface.
- **Sprint 3:** Implement OCR and PDF text extraction functionalities, integrate models into the app.
- **Subsequent Sprints:** Focus on testing, refining features, and incorporating user feedback.

3.2 DATA COLLECTION

High-quality datasets are essential for training robust detection models. The project utilizes two datasets to train models for detecting AI-generated text and images. Preprocessing ensures data consistency and quality.

3.2.1 CIFAKE Dataset for Images

- Algorithm: A Convolutional Neural Network (CNN) or Vision Transformer (ViT) will be used for binary classification to distinguish real from AI-generated images.
- Source: CIFAKE Dataset
- Attributes: Images with 32x32 pixels and 3 color channels (RGB).
- Classes: 2 (Real, AI-Generated). The original dataset includes 10 classes (Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck), but for this project, images are relabeled as "Real" (from CIFAR-10) or "AI-Generated" (created using Stable Diffusion v1-4).
- Number of Data Points: 120,000 (60,000 real, 60,000 AI-generated).

3.2.2 LLM - Detect AI-Generated Text Dataset for Text

- Algorithm: A transformer-based model, such as BERT or RoBERTa, will be fine-tuned for binary classification to identify human-written versus AI-generated text.

- Source: LLM Detect AI Generated Text Dataset
- Attributes: Text content and label (0 for human, 1 for AI-generated). Additional features may include TF-IDF with n-grams (3,5) and writing quality metrics (e.g., word length ratios, sentence length ratios).
- Classes: 2 (Human, AI-Generated).
- Number of Data Points: Approximately 57,000

3.3 MODEL DEVELOPMENT

Two machine learning models will be developed to detect AI-generated content, one for text and one for images, leveraging the datasets described above.

3.3.1 Text Detection Model

- Algorithm: A transformer-based NLP model, such as BERT or RoBERTa, will be fine-tuned for binary classification to distinguish human-written text from AI-generated text based on linguistic patterns.
- Training: The model will be trained on the LLM - Detect AI Generated Text Dataset, using preprocessed text data.
- Evaluation Metrics: Accuracy, precision, recall, and F1-score will be used to assess performance, with results summarized in a performance table.

3.3.2 Image Detection Model

- Algorithm: A CNN or Vision Transformer (ViT) will be trained to classify images as real or AI-generated by detecting artifacts or unnatural patterns.
- Training: The model will be trained on the CIFAKE Dataset, with images relabeled as “Real” or “AI-Generated” for binary classification.
- Evaluation Metrics: Accuracy, precision, recall, and F1-score will be used to evaluate performance.

3.4 APP DEVELOPMENT

The mobile app will be developed using Flutter, a cross-platform framework ensuring compatibility with Android and iOS. This approach optimizes development efficiency and provides a consistent user experience.

Key Features

- Text Input and Analysis: Users can paste text or upload PDFs, with text extraction handled by libraries like pdftron.
- Image Upload or Capture: Users can upload images or capture them via the device camera.
- Optical Character Recognition (OCR): OCR, implemented using libraries like Tesseract or Google Cloud Vision API, will extract text from images for analysis.
- Detailed Report: For each analysis, the app will display whether the content is AI-generated or human-created, with confidence scores and explanations (e.g., linguistic or visual features contributing to the classification).

Technologies

- Frontend: Flutter for cross-platform UI development.
- Backend: A server using Flask or FastAPI to host the models for inference.
- PDF Handling: Libraries like pdftron for extracting text from PDFs.
- OCR: Tesseract or Google Cloud Vision API for text extraction from images.

3.5 INTEGRATION AND TESTING

Model Integration

The trained text and image detection models will be integrated into the app. If model size or computational requirements are high, they will be deployed on a cloud server, with the app communicating via API calls. For on-device processing, models will be optimized (e.g., using quantization) to ensure efficiency.

API Integration

External APIs, such as those for OCR, will be integrated to enhance functionality. The app will handle API responses to provide seamless text extraction and analysis.

Testing

- **Unit Testing:** Individual components (e.g., model inference, OCR, PDF text extraction) will be tested for correctness.
- **Integration Testing:** Ensures all components (models, APIs, UI) work together seamlessly.
- **User Acceptance Testing (UAT):** Feedback from potential users will be collected to refine usability and functionality.

3.6 USER INTERFACE AND EXPERIENCE

The app will feature an intuitive, user-friendly interface designed to cater to both technical and non-technical users. Key screens include:

- **Home Screen:** Options for text input, PDF upload, image upload, or camera capture.
- **Analysis Screen:** Displays detection results, confidence scores, and explanations (e.g., “High perplexity suggests AI-generated text”).
- **Settings Screen:** Allows customization of preferences, such as notification settings.

Mockups will be created using tools like Figma to visualize the UI before implementation. Clear instructions and tooltips will enhance accessibility, ensuring a smooth user experience.

3.7 FIGURES AND TABLES

3.7.1 SYSTEM ARCHITECTURE

The system architecture is structured as a layered architecture, a common approach for mobile applications with AI components, as noted in resources like Mobile App Architecture Diagram. This design ensures separation of concerns, scalability, and maintainability, critical for an app handling complex AI detection tasks.

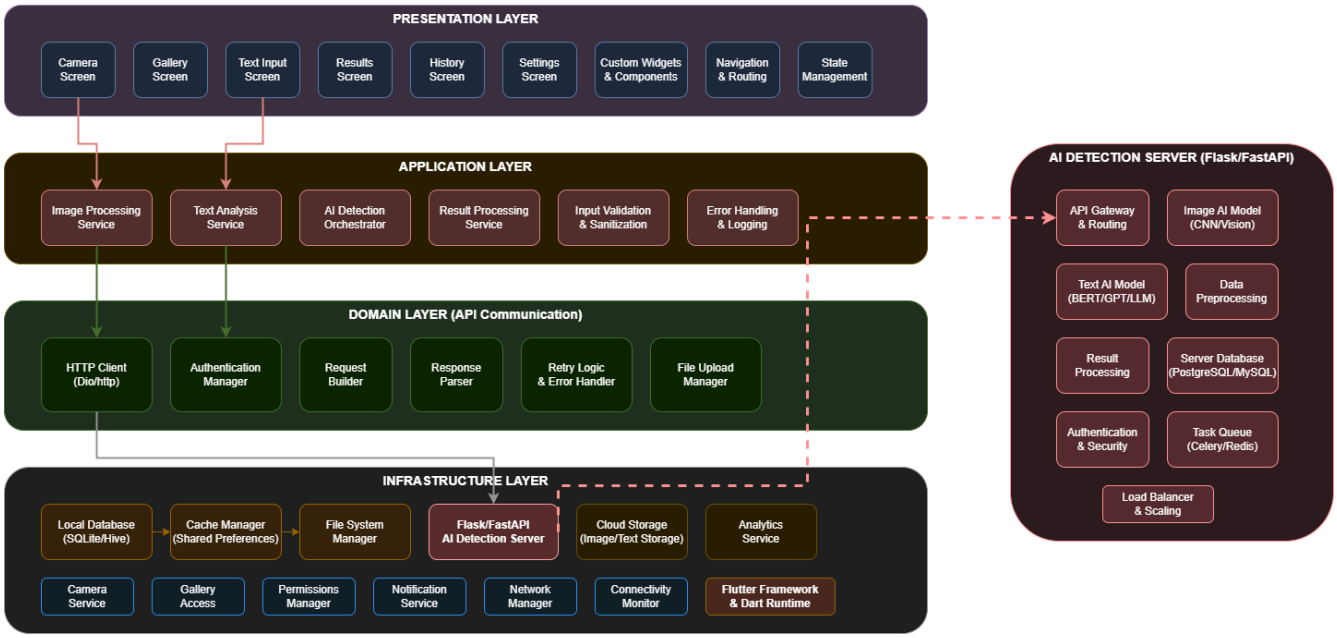

Figure 2: System Architecture

3.7.2 USE CASE DIAGRAM

The use case diagram outlines interactions for the same system, with actors User, Admin, and Server. Users can log in, recover passwords, submit text or images, and view results. Admins have extended capabilities, including authorizing actions, viewing logs, managing detection models, processing content (with optional extensions like comparing with AI patterns and analyzing metadata), and viewing accuracy metrics. The Server runs detection models to support content processing, highlighting distinct roles and functionalities.

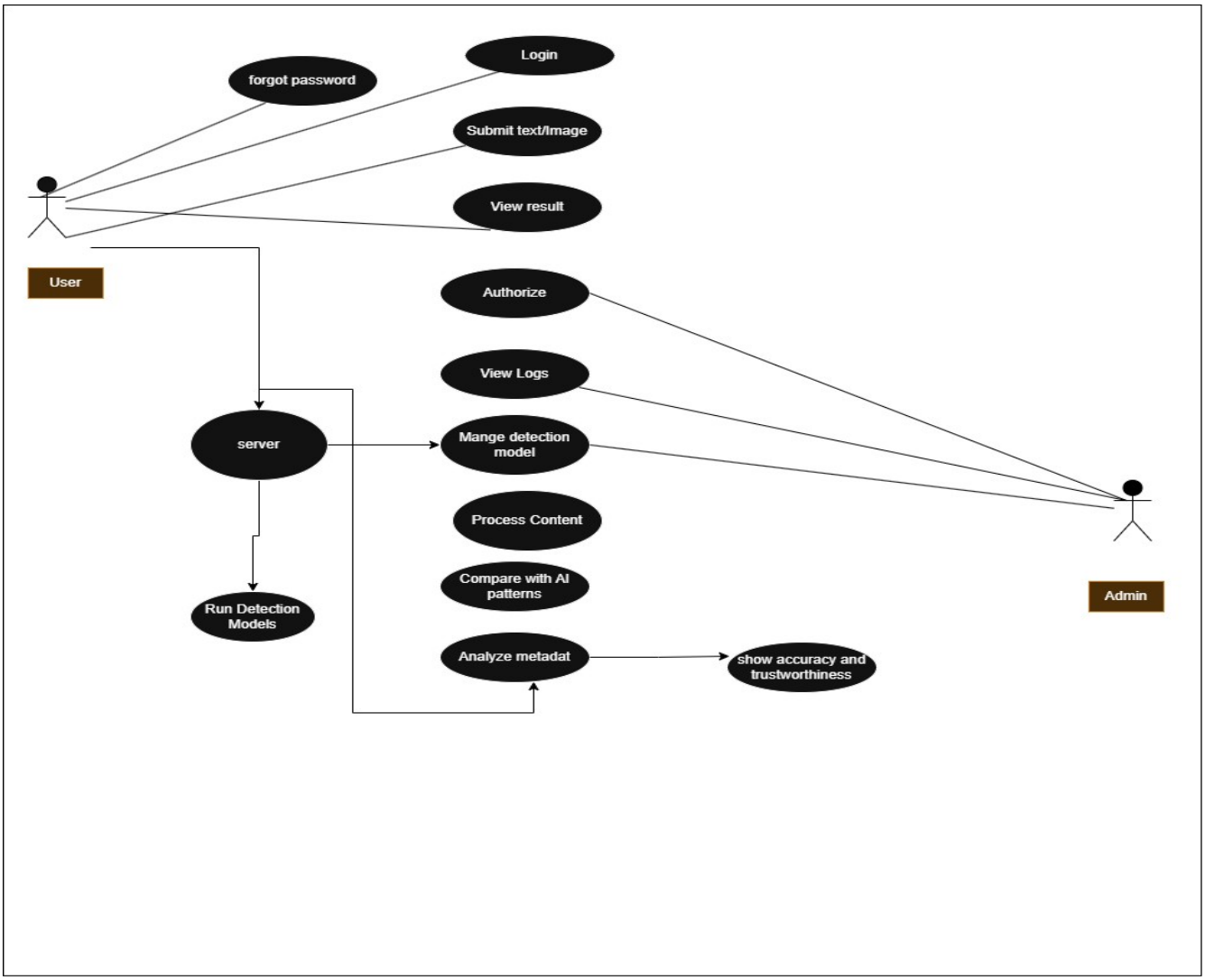

Figure 3: Use Case Diagram

4.PROPOSED DELIVERABLES

The project will deliver:

1. Mobile Application: A cross-platform app for Android and iOS, supporting text and image detection, PDF upload, and OCR.

2. Documentation:

- User Manual: Guides users on app usage.
- Technical Documentation: Details system architecture and models.

3. Research Report: Summarizes methodology, results, and significance in combating misinformation.

4. Deployment: App available on Google Play Store and Apple App Store.

5. PROJECT TASK AND SCHEDULE

5.1 PROJECT TASKS

• **Phase 1: Research and Planning** (2 weeks): Literature review, scope definition, and tool selection.

• **Phase 2: System Design** (2 weeks): Architecture and data collection planning.

• **Phase 3: Development** (4 weeks): Model development, API integration, and app development.

• **Phase 4: Testing and Evaluation** (2 weeks): Unit, integration, and user testing.

• **Phase 5: Deployment and Documentation** (4 weeks): App deployment and documentation.

5.2 GANTT CHART

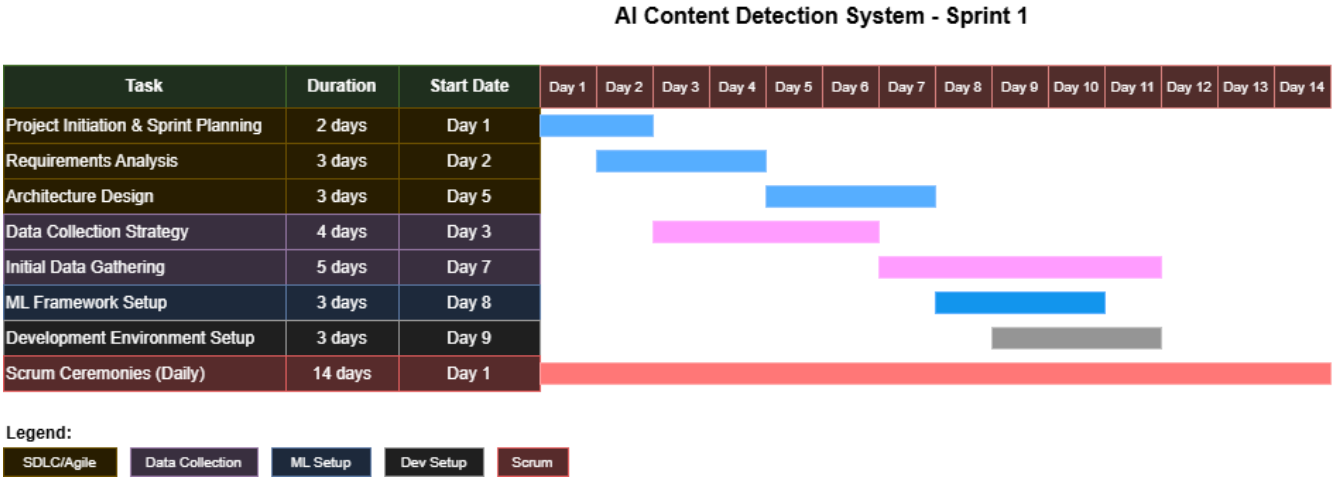

Figure 4: Sprint 1

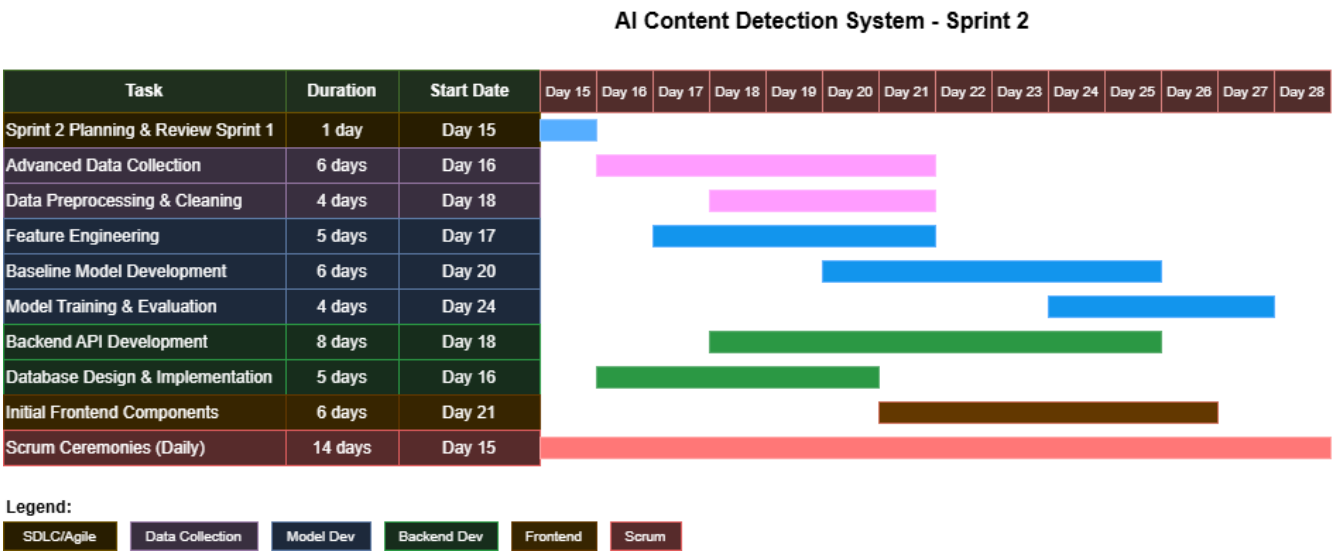

Figure 5: Sprint 2

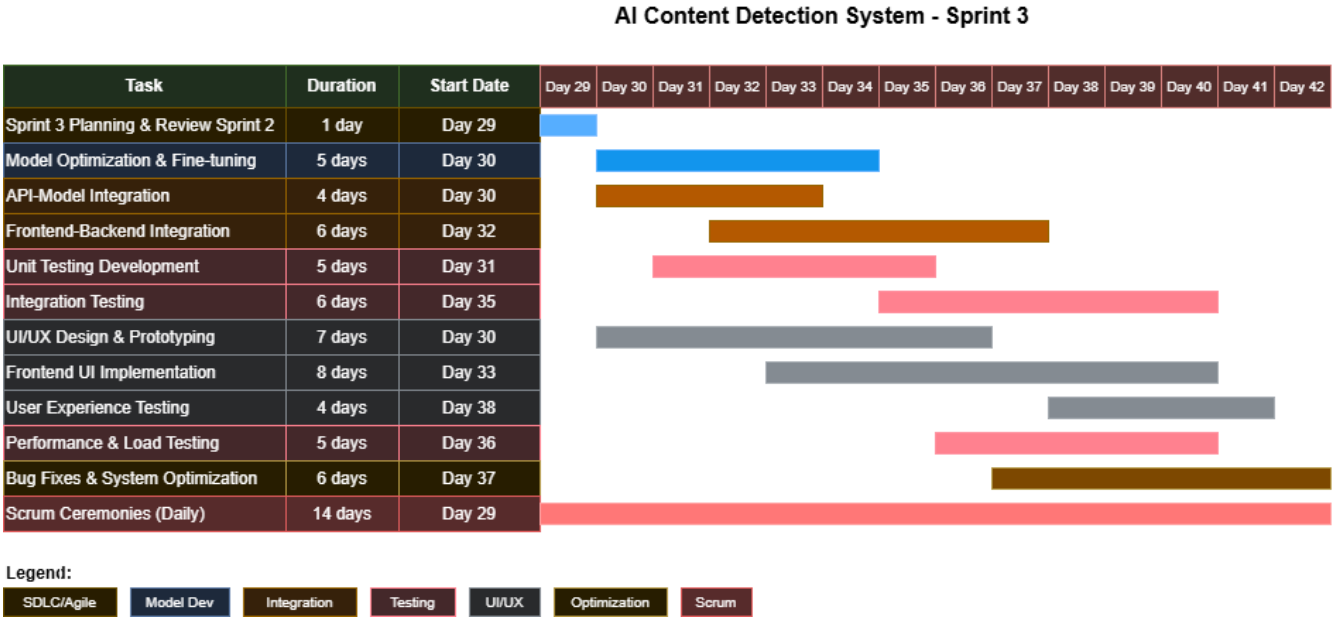

Figure 6: Sprint 3

REFERENCES

[1] OpenAI, "ChatGPT: Optimizing Language Models for Dialogue," [Online]. Available: https://openai.com/blog/chatgpt

[2] T. B. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877-1901, 2020.

[3] A. M. Elkhatat, "Evaluating the efficacy of AI content detection tools in differentiating between human and AI-generated text," *International Journal for Educational Integrity*, vol. 19, no. 1, pp. 1-16, 2023.

[4] J. Pu et al., "A Survey on LLM-Generated Text Detection: Necessity, Methods, and Future Directions," arXiv preprint arXiv:2404.12500, 2024.

[5] A. M. Elkhatat et al., "Testing of detection tools for AI-generated text," *International Journal for Educational Integrity*, vol. 19, no. 1, pp. 1-39, 2023.

[6] Winston AI, "The Most Trusted AI Detector," [Online]. Available: https://gowinston.ai

[7] Originality.ai, "AI Content Detector Accuracy Review," [Online]. Available: https://originality.ai

[8] Copyleaks, "AI Content Detection," [Online]. Available: https://copyleaks.com

[9] Grammarly, "AI Writing Assistance," [Online]. Available: https://grammarly.com

[10] J. J. Bird and A. Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," arXiv preprint arXiv:2303.14126, 2023. [Online]. Available: https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images

[11] S. Thite, "LLM - Detect AI Generated Text Dataset," [Online]. Available: https://www.kaggle.com/datasets/sunilthite/llm-detect-ai-generated-text-dataset